



Android App

Contexte

L'application Android à développer sert de pont entre l'utilisateur et le backend. L'app permet aux utilisateurs de s'authentifier via OAuth2, de générer des tokens API, et d'envoyer ces tokens au backend.

Comparaison des technologies



1. Android Natif

Simplicité et Robustesse : Java est un langage natif pour Android, ce qui permet une intégration direct avec le SDK Android. Le SDK c'est l'ensemble des outils mis à disposition pour aider les développeur à créer des apps pour des plateformes spécifiques (ici Android).

Le Java permet une implémentation rapide de l'auth, la gestion des tokens, les interations avec l'UI etc..

Accès aux API Android : Avec Java on a accès à tout les API Android natives, ça simplifie l'ajout des fonctionnalités

nécessaires.

Maintenance et Évolutivité : Java, en tant que langage historique d'Android, garantit une stabilité et une compatibilité solides pour les futures mises à jour. Cela assure que l'application restera fonctionnelle et facile à maintenir à long terme, sans avoir besoin de gros refactoring.

Communauté et Support : Java bénéficie d'une grande communauté de développeurs, ce qui rend la résolution des problèmes plus facile et permet une intégration aisée des bibliothèques existantes pour gérer des aspects comme l'authentification OAuth2 ou les requêtes REST vers le backend.

Conclusion : Android Natif en Java est idéal pour ce projet. Il fournit une solution robuste et simple pour développer une application légère, avec une maintenance facile et une compatibilité assurée pour les futures versions d'Android.



3. React Native

Multi-Plateforme : React Native permet de déployer l'application sur plusieurs plateformes.

Simplicité d'Implémentation : Pour une appli simple qui n'a pas besoin de fonctionnalités Android complexes, React Native est une solution rapide et efficace. Les fonctionnalités d'authentification et de gestion de tokens sont faciles à mettre

en place avec des bibliothèques JavaScript, bien supportées par la communauté.

Maintenance et Mises à Jour : La maintenance d'une app Android en React Native peut vite devenir compliqué à cause des variations entre versions Android et des limites des API. En plus, les performances ne sont pas toujours au top par rapport au natif.

Conclusion : Même si React Native est pratique pour du multi-plateforme, pour une simple appli Android comme celle-ci, il n'apporte pas grand-chose par rapport au natif. Une appli native est plus simple et plus efficace.



4. Flutter

Expérience Utilisateur Uniforme : Flutter permet de créer des interfaces utilisateur belles et uniformes sur Android et iOS. Toutefois, pour ce projet spécifique, où les exigences UI sont simples et ne nécessitent pas d'animations complexes ou de rendu graphique avancé, Flutter peut être excessif.

Développement Multi-Plateforme : Comme React Native, Flutter est intéressant pour un développement multi-plateforme. Cependant, pour une application Android uniquement, l'overhead introduit par Flutter en termes de taille de l'application et de complexité du développement n'est pas justifié.

Maintenance : Flutter en étant encore en pleine évolution, peut rendre la maintenance compliquée sur le long terme. Les mises à jour fréquentes peuvent forcer à retoucher le code souvent, ce qui n'est pas top pour une appli simple.

Conclusion : Flutter n'est pas le meilleur choix pour le projet où la simplicité et la légèreté sont de mise. Une solution native serait plus adaptée pour garantir une maintenance facile et une mise en œuvre rapide.

Conclusion Générale

Pour ce projet, où l'application Android sert surtout d'interface pour générer des tokens API et interagir simplement avec un backend, l'Android natif est la meilleure option, il apporte la simplicité, la robustesse et la compatibilité qu'il faut pour créer une appli légère, facile à maintenir, et bien intégrée à Android. Les frameworks multi-plateformes comme React Native ou Flutter sont moins utiles ici, car ils ajoutent de la complexité pour une appli Android toute simple.